

APPLICATION LAYER: WWW AND HTTP

- The **application layer** is the highest layer in the protocol suite.
- The application layer provides services to the user.
- Communication is provided using a logical connection, which means that the two application layers assume that there is an imaginary direct connection through which they can send and receive messages.
- The application layer is the only layer that provides services to the Internet user
- The application layer exchange messages with their peers on other machines
- Types of Application Protocols:
 - **Standard and Nonstandard Protocols**

Standard Application-Layer Protocols

- There are several application-layer protocols that have been standardized and documented by the Internet authority.
- Each standard protocol is a pair of computer programs that interact with the user and the transport layer to provide a specific service to the user.

Two very widely-used standardized application protocols:

- **SMTP : Simple Mail Transfer Protocol** is used to exchange electronic mail.
- **HTTP : Hyper Text Transport Protocol** is used to communicate between Web browsers and Web servers.

Nonstandard Application-Layer Protocols

- A programmer can create a nonstandard application-layer program if they can write two programs that provide service to the user by interacting with the transport layer.

APPLICATION-LAYER PARADIGMS

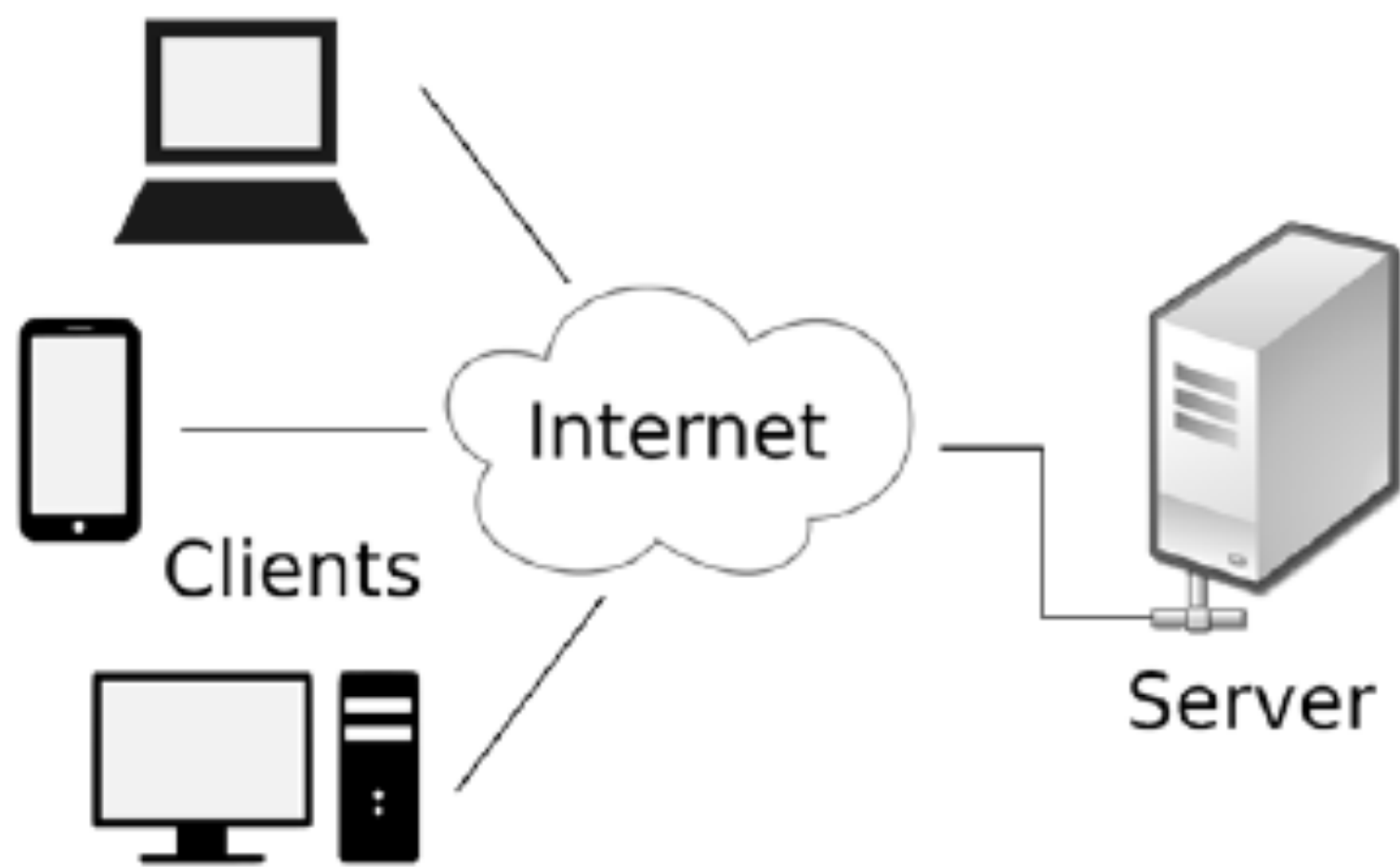
Two paradigms have been developed for Application Layer

1. Traditional Paradigm : Client-Server

2. New Paradigm : Peer-to-Peer

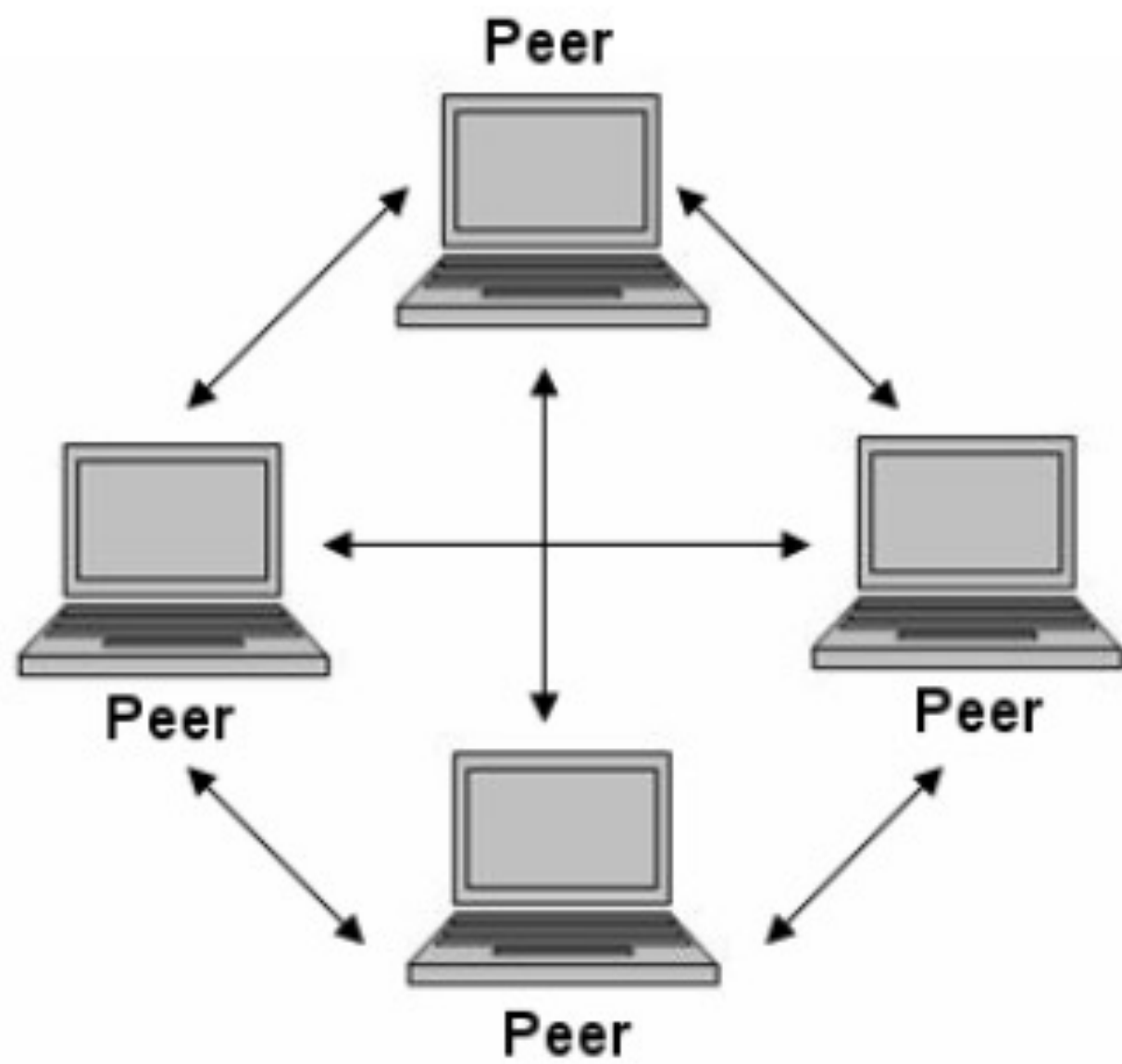
Client-Server Paradigm

- The **traditional paradigm** is called the client-server paradigm.
- In this paradigm, the service provider is an application program, called the ***server*** process; it runs continuously, waiting for another application program, called the ***client*** process, to make a connection through the Internet and ask for service.
- The server process must be running all the time; the client process is started when the client needs to receive service.



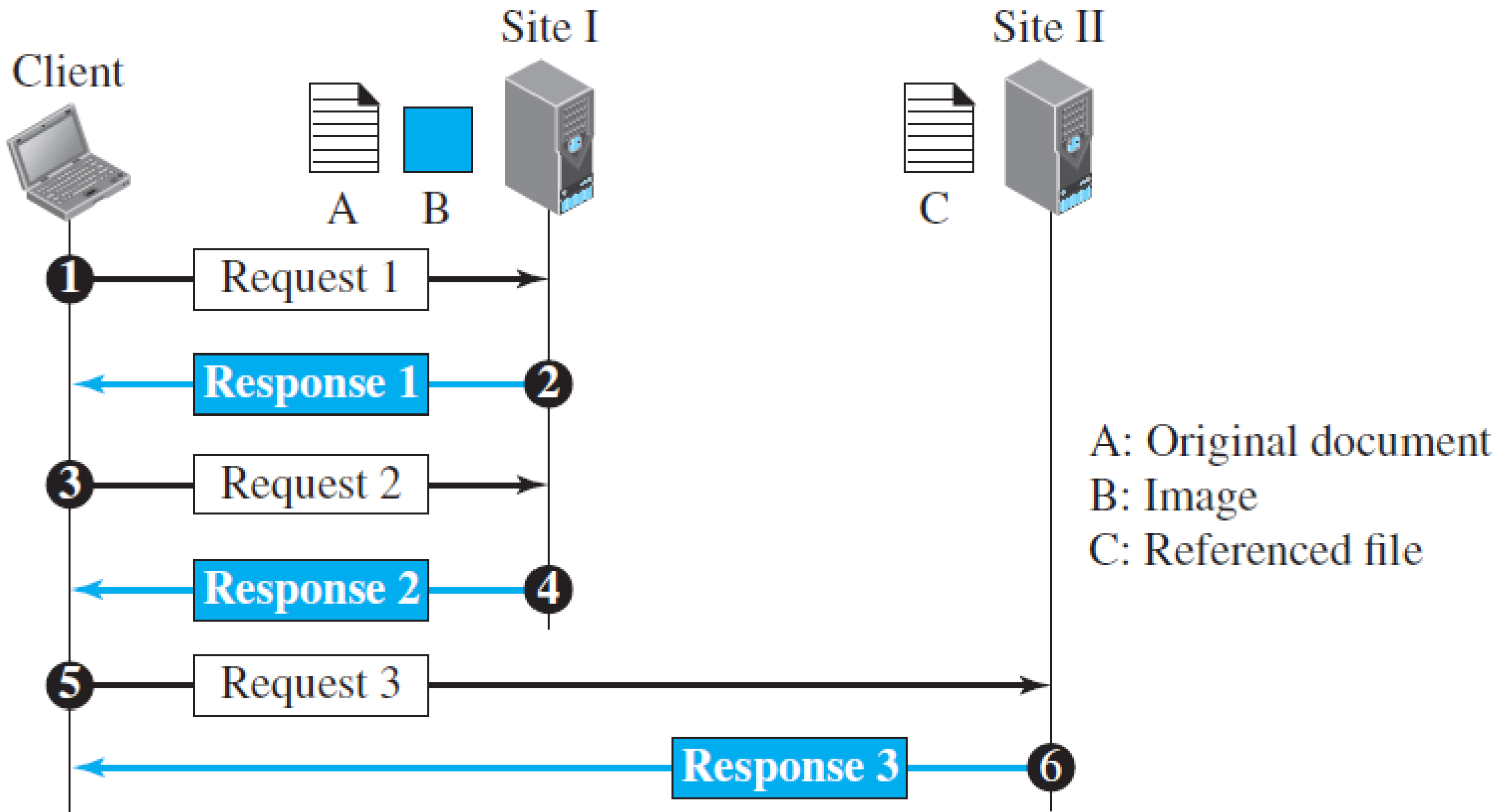
Peer-to-Peer(P2P) Paradigm

- A **new paradigm**, called the peer-to-peer paradigm has emerged to respond to the needs of some new applications.
- In this paradigm, there is no need for a server process to be running all the time and waiting for the client processes to connect.
- The responsibility is shared between peers.
- A computer connected to the Internet can provide service at one time and receive service at another time.
- A computer can even provide and receive services at the same time.



WWW (WORLD WIDE WEB)

- WWW is a distributed client/server service, in which a ***client*** (Browsers such as IE, Firefox, etc.) can access services at a ***server*** (Web server such as IIS, Apache).
- The service provided is distributed over many locations called sites.
- WWW was constructed originally by a small group of people led by Tim Berners Lee at CERN, in 1989 and in 1991 this was released to the world.



- This system allows document search and retrieval from any part of the Internet.
- The documents were having ***Hypertext*** as the content
- The units of information on the web can be referred to as pages, documents or **resources**.
- A document can contain text, images, sound and video, together called **Hypermedia**.
- Web is a vast collection of data, information, software and protocols , spread across the world in web servers, which are accessed by client machines by browsers through the Internet.

COMPONENTS OF THE WEB

- **Structural Components**

1. Web Clients/Browsers
2. Web Servers
3. Web Caches
4. Internet

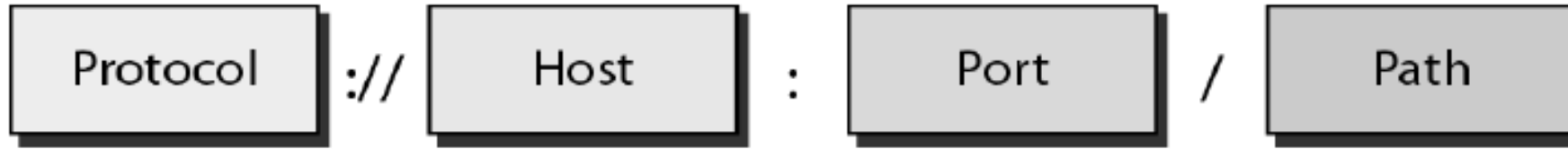
- **Semantic Components**

1. Hyper Text Transfer Protocol (HTTP)
2. Hyper Text Markup Language (HTML)
3. eXtensible Markup Language (XML)
4. Uniform Resource Identifier (URI)

- Clients use browser application to send URL's via HTTP to servers requesting a Web page.
- Web pages constructed using HTML /XML and consist of text, graphics, sounds plus embedded files, Servers (or caches) respond with requested Web page.
- Client's browser renders Web page returned by server
- Web Page is written using ***Hyper Text Markup Language (HTML)***.
- Displays text, graphics and sound in browser
- The entire system runs over standard networking protocols (TCP/IP, DNS)

URL - UNIFORM RESOURCE LOCATOR

- Uniform Resource Locator (URL), uniquely identify resources on the Internet
- URL provides information about its location on the Web
- When a user enters URL, browser forms a *request* message and sends it to the server.
- Web server retrieves the requested URL and sends back a *response* message.
- Web browser renders the response in HTML or appropriate format.
- **Format:** `http://www.domain_name/filename`
- **Example:** `http://www.cs.hello.org/index.html`



<http://www.mhhe.com/compsci/forouzan/>

The URL defines four parts –

- Method,
- Host computer,
- Port, and
- Path.

- **Method:** The method is the protocol used to retrieve the document from a server. For example, HTTP.
- **Host:** The host is the computer where the information is stored, and the computer is given an alias name. Web pages are mainly stored in the computers and the computers are given an alias name that begins with the characters "www". This field is not mandatory.
- **Port:** The URL can also contain the port number of the server, but it's an optional field. If the port number is included, then it must come between the host and path and it should be separated from the host by a colon.
- **Path:** Path is the pathname of the file where the information is stored. The path itself contain slashes that separate the directories from the subdirectories and files.

URI - Uniform Resource Identifiers

- URI is a string that identifies resources such as document, image, service, etc.
- It is of the form *scheme:scheme-specific*
- Scheme *identifies* a resource type, such as mailto for mail address, file for file name, etc. and scheme-specific is a *resource* identifier.
- Example is ***mailto: abc123@gmail.com***
- **URI** identifies a resource, whereas **URL** is used to locate a resource.

HTTP (HYPERTEXT TRANSFER PROTOCOL)

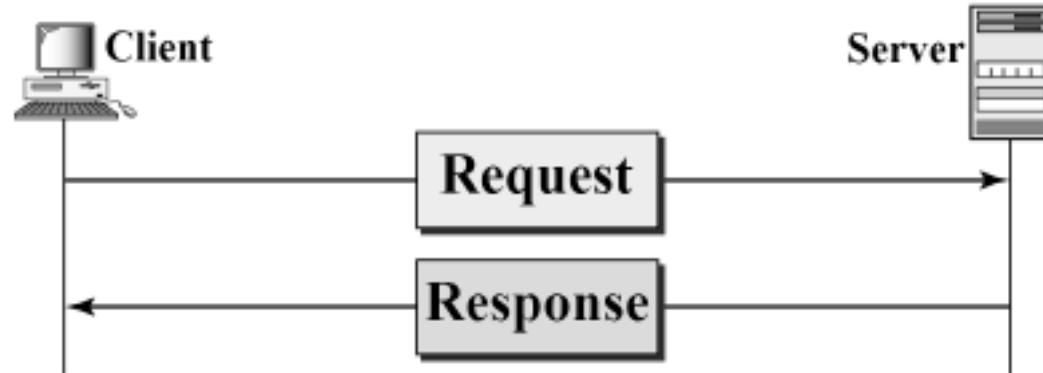
- The *HyperText Transfer Protocol (HTTP)* is used to define how the client server programs can be written to retrieve web pages from the Web.
- It is a protocol used to access the data on the World Wide Web (WWW).
- The HTTP protocol can be used to transfer the data in the form of plain text, hypertext, audio, video, and so on.
- HTTP is a **stateless request/response** protocol that governs client/server communication.

- An HTTP client sends a request; an HTTP server returns a response.
- The server uses the port number **80**; the client uses a temporary port number.
- HTTP uses the services of TCP, a connection-oriented and reliable protocol.
- HTTP is a text-oriented protocol. It contains *embedded* URL known as links.
- When hypertext is clicked, browser opens a new connection, retrieves file from the server and displays the file.

Features of HTTP

- **Connectionless protocol**
- **Media independent**
- **Stateless**

HTTP REQUEST AND RESPONSE MESSAGES



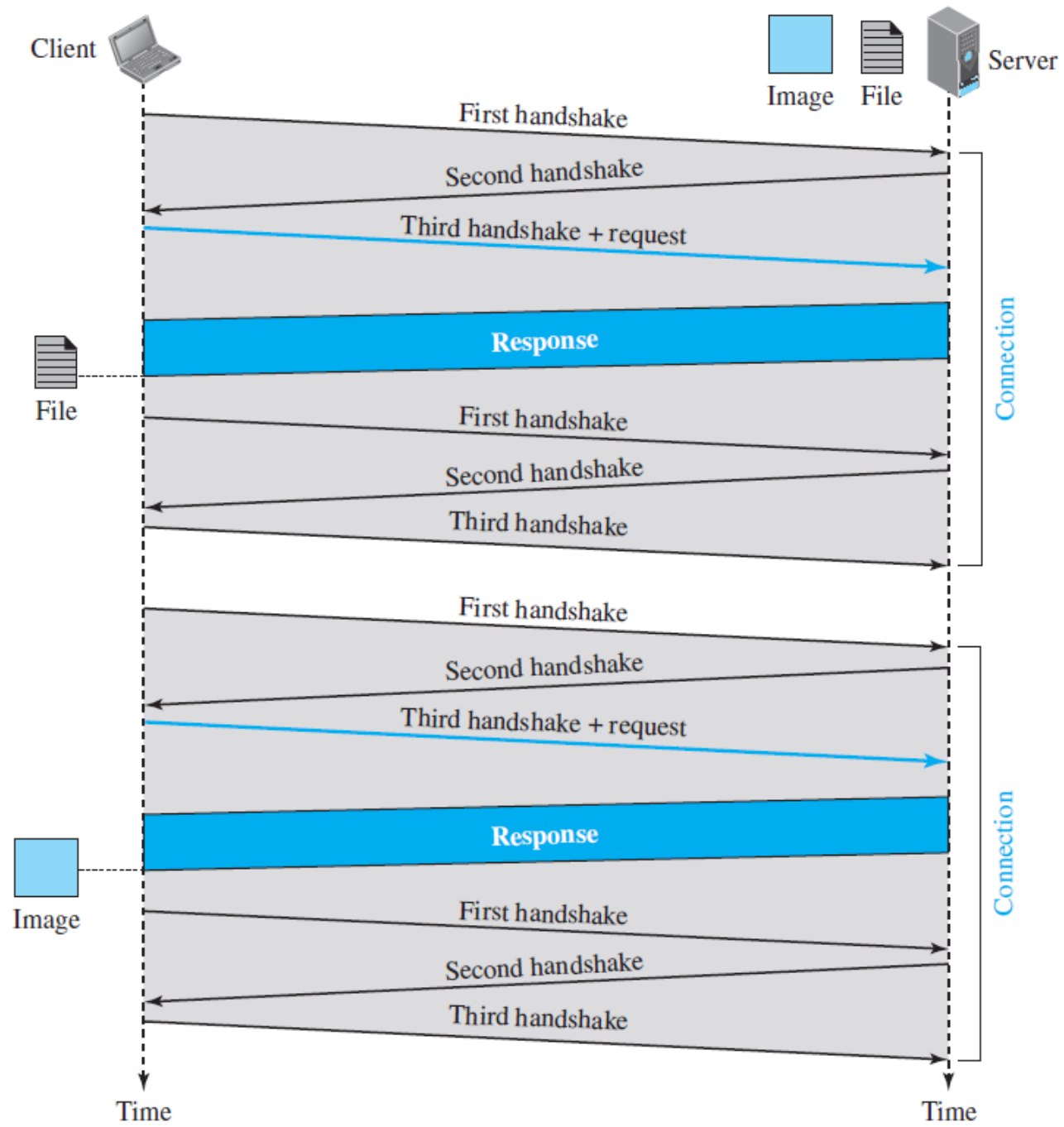
- **Request Message:** It is sent by the client that consists of a *request line*, *headers*, and sometimes a body.
- **Response Message:** The response message is sent by the server to the client that consists of a status line, headers, and sometimes a body.

Nonpersistent Connections

In a **nonpersistent connection**, one TCP connection is made for each request/response.

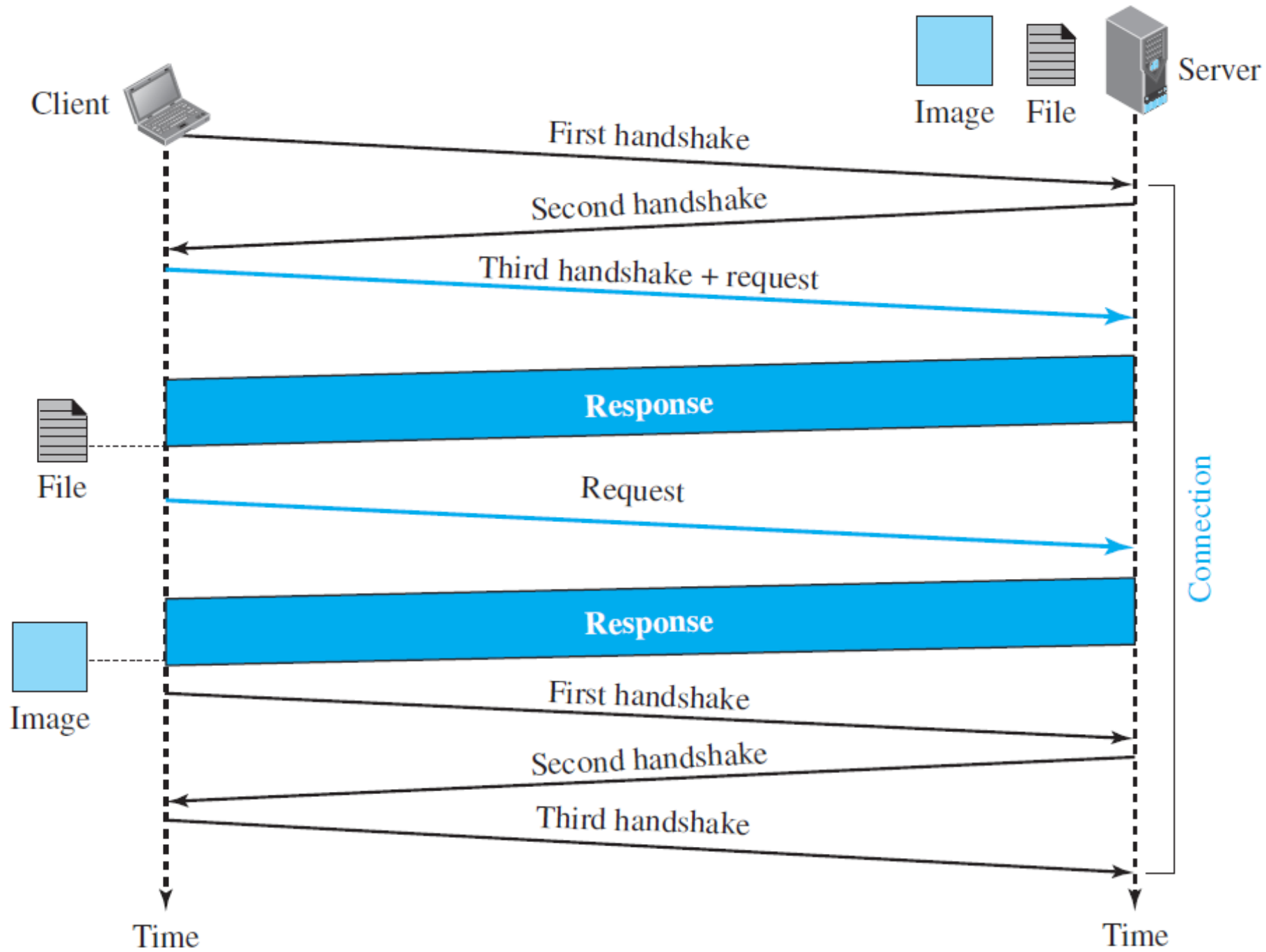
The following lists the steps in this strategy:

- 1.** The client opens a TCP connection and sends a request.
- 2.** The server sends the response and closes the connection.
- 3.** The client reads the data until it encounters an end-of-file marker; it then closes the connection.



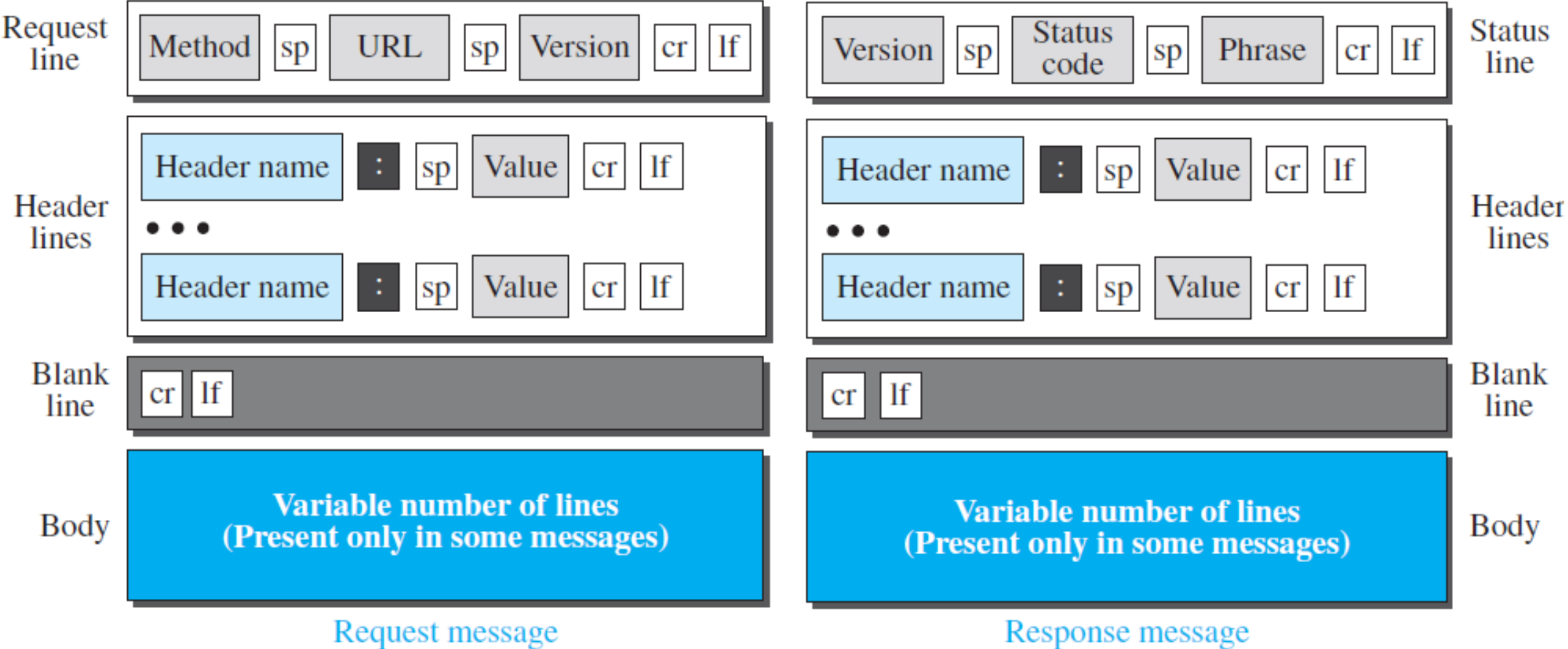
Persistent Connections

- In a ***persistent connection***, the server leaves the connection open for more requests after sending a response.
- The server can close the connection at the request of a client or if a time-out has been reached.
- The sender usually sends the length of the data with each response



Formats of the request and response messages

Legend sp: Space cr: Carriage Return lf: Line Feed



Methods

<i>Method</i>	<i>Action</i>
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

